CPU SCHEDULING IN OPERATING SYSTEM

CPU scheduling is a process used by the operating system to decide which task or process gets to use the CPU at a particular time. This is important because a CPU can only handle one task at a time, but there are usually many tasks that need to be processed.

The following are different purposes of a CPU scheduling time.

- Maximize the CPU utilization.
- Minimize the response and waiting time of the process.

What is the Need for a CPU Scheduling Algorithm?

CPU scheduling is the process of deciding which process will own the CPU to use while another process is suspended. The main function of CPU scheduling is to ensure that whenever the CPU remains idle, the OS has at least selected one of the processes available in the ready-to-use line.

Terminologies Used in CPU Scheduling:

1.Arrival Time: The time at which the process arrives in the ready queue.

2.Completion Time: The time at which the process completes its execution.

3.Burst Time: Time required by a process for CPU execution.

4.Turn Around Time: Time Difference between completion time and arrival time.

$$Turn\ Around\ Time = Completion\ Time - Arrival\ Time$$

5.Waiting Time(W.T): Time Difference between turn around time and burst time.

$$Waiting\ Time = Turn\ Around\ Time - Burst\ Time$$

6.Gantt Chart:A Gantt Chart is a graphical representation of the execution order of processes on the CPU over time.The horizontal axis represents time.Each block

represents a process.The length of the block corresponds to its burst time.It helps determine Completion Time, Turnaround Time, and Waiting Time.

Example:

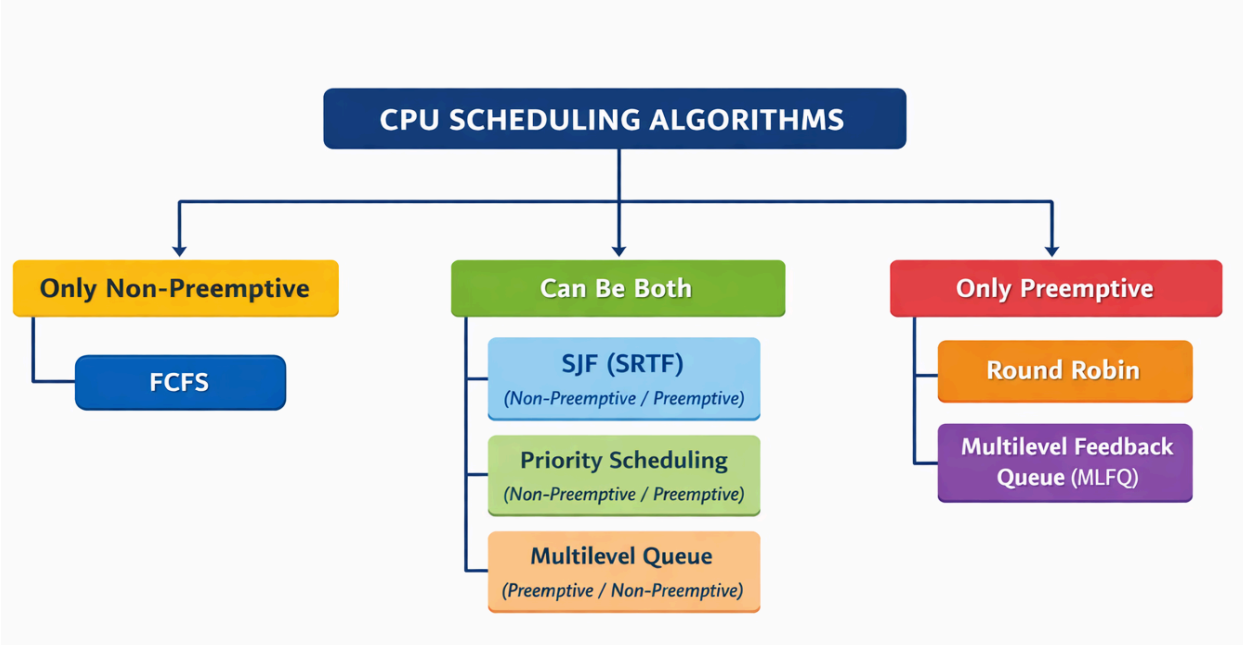
P1	P2	P3	P4

0 3 7 9 11

Different Types of CPU Scheduling Algorithms

There are mainly two types of scheduling methods:

- **Preemptive Scheduling:** Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.
- **Non-Preemptive Scheduling:** Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.



1.First Come First Serve(FCFS):

First Come, First Serve (FCFS) is the simplest CPU scheduling algorithm. Processes are executed in the order they arrive in the ready queue, just like people standing in a line. FCFS is a **non-preemptive** scheduling algorithm, which means once a process starts execution, it continues until it finishes or goes for I/O. The CPU does not interrupt it for another process.

How does FCFS Work?

1. Processes enter the ready queue based on their arrival time.

2. If all processes arrive at the same time, they are executed in the order given (P1, P2, P3, …).

3. If processes arrive at different times, the CPU always selects the process that arrived earliest among the available processes.

4. Once a process starts execution, it runs until completion.

5. The CPU then selects the next process in arrival order.

Example:Consider the following processes with their Arrival Time (AT) and Burst Time (BT).Using the FCFS scheduling algorithm calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	**Arrival Time (AT)**	**Burst Time (BT)**
P1	1	5
P2	0	3
P3	2	6
P4	4	2

Step 1: Arrange According to Arrival Time

Sort by Arrival Time:P2 (0) → P1 (1) → P3 (2) → P4 (4)

Execution Order:P2 → P1 → P3 → P4

Step 2:Draw Gantt Chart

P2	P1	P3	P4

0 **3** **8** **14** **16**

Step 3:Calculate TAT and WT using formula

Process	**AT**	**BT**	**CT**	**TAT = CT − AT**	**WT = TAT − BT**
P2	0	3	3	3 − 0 = 3	3 − 3 = 0
P1	1	5	8	8 − 1 = 7	7 − 5 = 2
P3	2	6	14	14 − 2 = 12	12 − 6 = 6
P4	4	2	16	16 − 4 = 12	12 − 2 = 10

Step 4:Calculate Average TAT and Average WT

Average Turnaround Time = (3 + 7 + 12 + 12) / 4 = 8.5

Average Waiting Time = (0 + 2 + 6 + 10) / 4 = 4.5

Advantages of FCFS: Simple and easy to implement and no starvation, as processes are executed in the order of arrival.

Disadvantages of FCFS:High average waiting time and suffers from the convoy effect, where short processes wait behind long processes.

2.Shortest Job First(SJF):

Shortest Job First (SJF) is a CPU scheduling algorithm in which the process with the smallest burst time is selected for execution next. It reduces the average waiting time compared to FCFS.

SJF can be non-preemptive or preemptive.
In non-preemptive SJF, once a process starts execution, it runs until completion. In preemptive SJF (Shortest Remaining Time First – SRTF), the currently running process can be interrupted if a new process arrives with a shorter remaining burst time.

How does SJF Work?

1. Processes enter the ready queue based on their arrival time.
2. If all processes arrive at the same time, the process with the smallest burst time is selected first.
3. If processes arrive at different times, among the available processes in the ready queue, the one with the shortest burst time is selected.
4. In non-preemptive SJF, once a process starts execution, it runs until completion.
5. The CPU then selects the next shortest job from the remaining processes.

Example 1: Consider the following processes with their Arrival Time (AT) and Burst Time (BT).Using the **SJF non-preemptive** scheduling algorithm calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	AT	BT
P1	0	6
P2	1	4
P3	2	2
P4	3	5

Step 1: Decide Execution Order

At time 0,
Only P1 is available → Execute P1.

At time 6,
P2, P3 and P4 are available.
Shortest burst time = P3 (2) → Execute P3.

Remaining: P2 (4), P4 (5)
Shortest = P2 → Execute P2.

Finally execute P4.

Execution Order:
P1 → P3 → P2 → P4

Step 2:Draw Gantt Chart

P1	P3	P2	P4

0 6 8 12 17

Step 3:Calculate TAT and WT

Process	**AT**	**BT**	**CT**	**TAT = CT − AT**	**WT = TAT − BT**
P1	0	6	6	6 − 0 = 6	6 − 6 = 0
P3	2	2	8	8 − 2 = 6	6 − 2 = 4
P2	1	4	12	12 − 1 = 11	11 − 4 = 7
P4	3	5	17	17 − 3 = 14	14 − 5 = 9

Step 4:Calculate Averages

Average Turnaround Time
=> (6 + 6 + 11 + 14) / 4 = 9.25

Average Waiting Time
=> (0 + 4 + 7 + 9) / 4 = 5

Example 2:Consider the following processes with their Arrival Time (AT) and Burst Time (BT).Using the **SRTF(SJF-preemptive)** scheduling algorithm calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	AT	BT
P1	0	7
P2	1	4
P3	1	2
P4	3	2
P5	5	4

Step 1: Decide Execution Order (Time-wise)

At time 0,
Only P1 available → Execute P1.

At time 1,
P2 (4) and P3 (2) arrive.
Remaining of P1 = 6.
Shortest = P3 → Preempt P1.

At time 3,
P3 completes.
Available: P1 (6), P2 (4), P4 (2).
Shortest = P4 → Execute P4.

At time 5,
P4 completes.
P5 arrives (4).
Available: P1 (6), P2 (4), P5 (4).
Equal burst time (4 & 4) → Choose earlier arrival → P2.

At time 9,
P2 completes.

Remaining: P1 (6), P5 (4).
Shortest = P5 → Execute P5.

At time 13,
P5 completes.
Finally execute P1.

At time 19,
P1 completes.

Step 2:Gantt Chart

P1	P3	P4	P2	P5	P1

0 1 3 5 9 13 19

Step 3:Calculate TAT and WT

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	7	19	19 − 0 = 19	19 − 7 = 12
P2	1	4	9	9 − 1 = 8	8 − 4 = 4
P3	1	2	3	3 − 1 = 2	2 − 2 = 0
P4	3	2	5	5 − 3 = 2	2 − 2 = 0
P5	5	4	13	13 − 5 = 8	8 − 4 = 4

Step 4:Calculate Averages

Average Turnaround Time
=> (19 + 8 + 2 + 2 + 8) / 5= 7.8

Average Waiting Time
=>(12 + 4 + 0 + 0 + 4) / 5 = 4

Advantages of SJF: Minimum average waiting time compared to many other scheduling algorithms and efficient for systems where shorter jobs are preferred.

Disadvantages of SJF: Difficult to predict burst time in real systems and can cause starvation of longer processes.

3.Priority Scheduling

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority value. The process with the highest priority is selected for execution first. Priority Scheduling can be non-preemptive or preemptive. (Assume smaller number indicates higher priority.)

How does it Work?

1. Each process is assigned a priority value before execution.
2. Processes enter the ready queue based on their arrival time.
3. At any given time, the CPU selects the process with the highest priority from the ready queue.
4. If two processes have the same priority, the process that arrived earlier is selected(FCFS).
5. If the algorithm is **non-preemptive**, once a process starts execution, it runs until completion.
6. If the algorithm is **preemptive**, whenever a new process arrives with a higher priority than the currently running process, the current process is preempted and the higher priority process is executed.
7. The scheduling continues until all processes complete execution.

In preemptive Priority Scheduling, a running process can be interrupted if a newly arrived process has a higher priority.

(Note: Assume lower number indicates higher priority.)

Example 1:Consider the following processes with Arrival Time (AT), Burst Time (BT), and Priority (P).Using the **Non-Preemptive Priority** Scheduling algorithm calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	AT	BT	Priority
P1	0	5	3
P2	1	3	1
P3	2	4	4
P4	3	2	2

Step 1: Decide Execution Order

At time 0,
Only P1 is available → Execute P1.

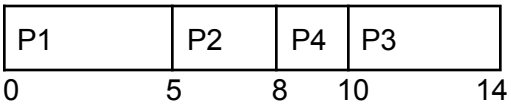
At time 5,
P2, P3 and P4 are available.
Highest priority = P2 (1) → Execute P2.

Remaining: P3 (4), P4 (2)
Highest priority = P4 → Execute P4.

Finally execute P3.

Execution Order:
P1 → P2 → P4 → P3

Step 2: Draw Gantt Chart



Step 3: Calculate TAT and WT

Process	**AT**	**BT**	**CT**	**TAT = CT − AT**	**WT = TAT − BT**
P1	0	5	5	5 − 0 = 5	5 − 5 = 0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4
P4	3	2	10	10 − 3 = 7	7 − 2 = 5
P3	2	4	14	14 − 2 = 12	12 − 4 = 8

Step 4:Calculate Averages

Average Turnaround Time
= >(5 + 7 + 7 + 12) / 4 = 7.75

Average Waiting Time
=> (0 + 4 + 5 + 8) / 4= 4.25

Example 2:Consider the following processes with Arrival Time (AT), Burst Time (BT), and Priority (P).Using the **Preemptive Priority** Scheduling algorithm calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	AT	BT	Priority
P1	0	6	3
P2	1	4	2
P3	1	3	2
P4	2	2	1
P5	4	3	3

Step 1: Decide Execution Order (Time-wise)

At time 0
Only P1 available → Execute P1.

At time 1
P2 and P3 arrive (priority 2).
Both have higher priority than P1 (3).
Same priority → choose earlier in queue → P2.
Preempt P1 → Execute P2.

At time 2
P4 arrives (priority 1).
Higher priority than P2 → Preempt P2 → Execute P4.

At time 4
P4 completes.

Available:
P1 (remaining 5),
P2 (remaining 3),
P3 (3),
P5 (3).

Highest priority = P2 and P3 (priority 2).
Same priority → Same arrival time → Choose which has lower process Id->P2->Execute P2.

At time 7
P2 completes.

Next highest priority = P3 → Execute P3.

At time 10
P3 completes.

Remaining: P1 (5), P5 (3).
Same priority (3).
FCFS → P1 executes first.

At time 15
P1 completes.

Finally execute P5.

At time 18
P5 completes.

Step 2:Draw Gantt Chart

P1	P2	P4	P2	P3	P1	P5

0 1 2 4 7 10 15 18

Step 3:Calculate TAT and WT

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	6	15	15 − 0 = 15	15 − 6 = 9
P2	1	4	7	7 − 1 = 6	6 − 4 = 2
P3	1	3	10	10 − 1 = 9	9 − 3 = 6
P4	2	2	4	4 − 2 = 2	2 − 2 = 0
P5	4	3	18	18 − 4 = 14	14 − 3 = 11

Step 4:Calculate Averages

Average Turnaround Time
= (15 + 6 + 9 + 2 + 14) / 5
= 46 / 5
= 9.2

Average Waiting Time
= (9 + 2 + 6 + 0 + 11) / 5
= 28 / 5
= 5.6

Advantages of Priority Scheduling: Priority Scheduling ensures that important or critical processes are executed first and it is suitable for systems where task importance must be considered. It provides flexibility by allowing the operating system to control execution order based on assigned priority levels.

Disadvantages of Priority Scheduling: Priority Scheduling may cause starvation of low-priority processes and it requires careful priority assignment to avoid unfair scheduling.

4.Round Robin

Round Robin is a preemptive CPU scheduling algorithm designed for time-sharing systems. In this algorithm, each process is assigned a fixed time quantum (time slice), and the CPU is allocated to processes in cyclic order. If a process does not complete within its time quantum, it is preempted and placed at the end of the ready queue.

How does it Work?

1. Processes enter the ready queue based on their arrival time.

2. The CPU selects the first process from the ready queue and executes it for one time quantum.

3. If the process completes within the time quantum, it is removed from the system.

4. If the process does not complete, it is preempted after the time quantum expires and placed at the end of the ready queue.

5. Newly arriving processes are added to the end of the ready queue.

6. The CPU continues to execute processes in cyclic order until all processes complete execution.

Example:Consider the following processes with Arrival Time (AT) and Burst Time (BT).Using Round Robin Scheduling (Time Quantum = 3), calculate CT, TAT, WT, and averages.

Process	AT	BT
P1	1	5
P2	2	3
P3	2	6
P4	5	2
P5	7	4

Step 1: Decide Execution Order (Time-wise)

At time 0
CPU is idle.
Ready Queue: —

At time 1
P1 arrives → Execute P1.
Ready Queue: —

At time 2
P2 and P3 arrive (same time).
Insert according to Process ID order → P2, P3.
Ready Queue: P2, P3

At time 4
Time quantum expires for P1 (remaining 2).

Insert P1 at end of queue.
Execute P2.
Ready Queue: P3, P1

At time 5
P4 arrives → Insert at end.
Ready Queue: P3, P1, P4

At time 7
P2 completes.
P5 arrives at time 7 → Insert at end.
Execute P3.
Ready Queue: P1, P4, P5

At time 10
Time quantum expires for P3 (remaining 3).
Insert P3 at end.
Execute P1.
Ready Queue: P4, P5, P3

At time 12
P1 completes.
Execute P4.
Ready Queue: P5, P3

At time 14
P4 completes.
Execute P5.
Ready Queue: P3

At time 17
Time quantum expires for P5 (remaining 1).
Insert P5 at end.
Execute P3.
Ready Queue: P5

At time 20
P3 completes.
Execute P5.
Ready Queue: —

At time 21
P5 completes.

Execution Order:
Idle → P1 → P2 → P3 → P1 → P4 → P5 → P3 → P5

Step 2: Draw Gantt Chart

	P1	P2	P3	P1	P4	P5	P3	P5

0 1 4 7 10 12 14 17 20 21

Step 3:Calculate TAT and WT

Process	**AT**	**BT**	**CT**	**TAT = CT − AT**	**WT = TAT − BT**
P1	1	5	12	12 − 1 = 11	11 − 5 = 6
P2	2	3	7	7 − 2 = 5	5 − 3 = 2
P3	2	6	20	20 − 2 = 18	18 − 6 = 12
P4	5	2	14	14 − 5 = 9	9 − 2 = 7
P5	7	4	21	21 − 7 = 14	14 − 4 = 10

Step 4:Calculate Averages

Average Turnaround Time
=> (11 + 5 + 18 + 9 + 14) / 5 = 11.4

Average Waiting Time
=> (6 + 2 + 12 + 7 + 10) / 5 = 7.4

Advantages of Round Robin: Round Robin ensures fair CPU allocation because each process gets equal execution time and it prevents starvation. It is suitable for time-sharing systems where quick response is important.

Disadvantages of Round Robin: Round Robin can cause high context switching overhead if the time quantum is too small and it may increase waiting time if the time quantum is too large.

5.Multilevel Queue Scheduling

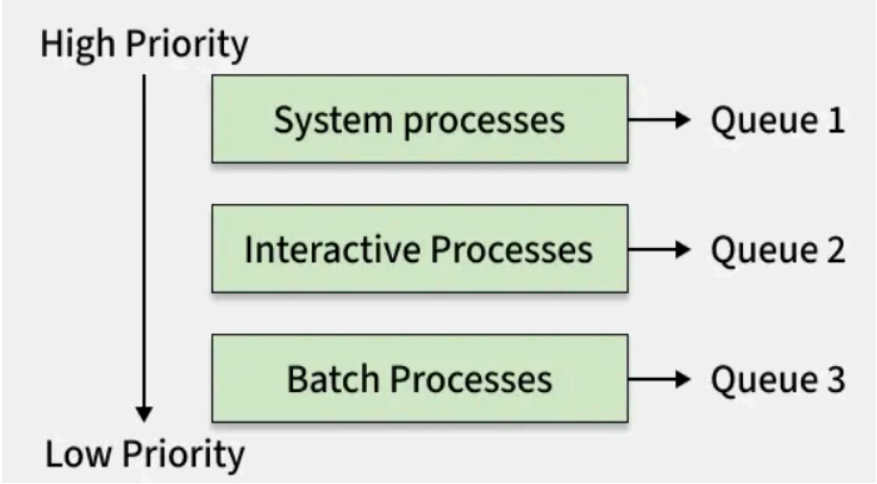
Multilevel Queue Scheduling is a CPU scheduling algorithm in which the ready queue is divided into multiple separate queues based on process type or priority class. Each queue has its own scheduling algorithm, and processes are permanently assigned to one queue. A process does not move between queues once assigned.

Scheduling occurs at two levels:

- Scheduling within each queue
- Scheduling between the queues

Multilevel Queue Scheduling can be either non-preemptive or preemptive depending on how scheduling between queues is defined.

Common classification:



Each queue can use a different scheduling algorithm (for example, Round Robin for interactive processes and FCFS for batch processes).

Between the queues, scheduling can be done using:

- Fixed priority (higher queue executes first)
  OR
- Time slicing between queues

How does it Work?

1. The ready queue is divided into multiple separate queues based on process category or priority level.

2. Each queue uses its own scheduling algorithm such as FCFS, Round Robin, or Priority Scheduling.

3. Scheduling between queues is usually done using fixed priority, where one queue has higher priority over others.

4. The CPU selects the highest priority non-empty queue for execution.

5. If scheduling between queues is non-preemptive, once a process from a lower priority queue starts execution, it runs until completion even if a higher priority process arrives.

6. If scheduling between queues is preemptive, a running process from a lower priority queue is immediately interrupted when a higher priority queue process arrives.

7. The scheduling process continues until all processes complete execution.

Example:Consider the following processes with Arrival Time (AT), Burst Time (BT), and Queue Type.The system uses **Multilevel Queue Scheduling** with the following conditions:

• There are two queues:
System Queue (Higher Priority)
User Queue (Lower Priority)

• Scheduling between queues is **Non-Preemptive**.

• The System Queue uses **Round Robin Scheduling (Time Quantum = 2)**.

• The User Queue uses **FCFS Scheduling**.Using the Non-Preemptive Multilevel Queue Scheduling algorithm, calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and the average Turnaround Time and average Waiting Time.

Process	AT	BT	Queue
P1	0	4	User
P2	1	3	System
P3	2	2	User
P4	3	2	System

Step 1: Decide Execution Order (Time-wise)

At time 0
P1 (User) arrives → Execute P1.

At time 1
P2 (System) arrives but scheduling is non-preemptive → P1 continues.

At time 4
P1 completes.
Now System Queue has higher priority → Execute P2 (RR).

At time 6
Time quantum expires for P2 (remaining 1) → Execute P4.

At time 8
P4 completes → Execute remaining P2.

At time 9
P2 completes → Execute User Queue (FCFS).

At time 9–11
Execute P3.

Execution Order:
P1 → P2 → P4 → P2 → P3

Step 2:Gantt Chart

P1	P2	P4	P2	P3

0 4 6 8 9 11

Step 3:Calculate CT,TAT,WT

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	4	4	4	0
P2	1	3	9	8	5
P3	2	2	11	9	7
P4	3	2	8	5	3

Step 4:Calculate Averages

Average Turnaround Time
= (4 + 8 + 9 + 5) / 4
= 26 / 4
= 6.5

Average Waiting Time
= (0 + 5 + 7 + 3) / 4
= 15 / 4
= 3.75

Example 2: Consider the following processes with Arrival Time (AT), Burst Time (BT), and Queue Type.The system uses Multilevel Queue Scheduling with the following conditions:

• Two queues:
  System Queue (Higher Priority)
  User Queue (Lower Priority)

• Scheduling between queues is **Preemptive**.

• System Queue uses Round Robin (Time Quantum = 2).

• User Queue uses FCFS.

• A higher priority queue process immediately preempts a lower priority queue process.

Using Preemptive Multilevel Queue Scheduling, calculate CT, TAT, WT and the averages.

Process	AT	BT	Queue
P1	0	4	User
P2	1	3	System
P3	2	2	User
P4	3	2	System

Step 1: Decide Execution Order (Time-wise)

At time 0
P1 (User) arrives → Execute P1.

At time 1
P2 (System) arrives → Higher priority → Preempt P1.
Execute P2.

At time 3
Time quantum expires for P2 (remaining 1).
Execute P4.

At time 5
P4 completes → Execute remaining P2.

At time 6
P2 completes → Execute User Queue (FCFS).

At time 6–9
Execute remaining P1.

At time 9–11
Execute P3.

Execution Order:
P1 → P2 → P4 → P2 → P1 → P3

Step 2: Gantt Chart

P1	P2	P4	P2	P1	P3

0 1 3 5 6 9 11

Step 3:Calculate CT,TAT,WT

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	4	9	9	5
P2	1	3	6	5	2
P3	2	2	11	9	7
P4	3	2	5	2	0

Step 4:Calculate Averages

Average Turnaround Time
= (9 + 5 + 9 + 2) / 4
= 25 / 4
= 6.25

Average Waiting Time
= (5 + 2 + 7 + 0) / 4
= 14 / 4
= 3.5

Advantages of Multilevel Queue Scheduling:It allows separation of processes based on type and it supports different scheduling strategies for different process categories.

Disadvantages of Multilevel Queue Scheduling:Lower priority queues may suffer starvation and processes cannot move between queues once assigned.

6.Multilevel Feedback Queue Scheduling:

Multilevel Feedback Queue Scheduling is an advanced CPU scheduling algorithm in which the ready queue is divided into multiple priority levels. Unlike Multilevel Queue Scheduling, processes are not permanently assigned to one queue. A process can move between queues based on its execution behavior.

It is a preemptive scheduling algorithm, meaning if a higher priority queue process is ready, it can preempt a currently running lower priority process.This algorithm is designed to favor short or interactive processes while preventing starvation.

How does it Work?

1. The ready queue is divided into multiple queues with different priority levels.

2. Each queue has its own scheduling algorithm, commonly Round Robin with different time quantums.

3. All new processes enter the highest priority queue.

4. The CPU always selects the highest priority non-empty queue.

5. If a process uses its entire time quantum, it is demoted to the next lower priority queue.

6. If a process completes within its time quantum, it leaves the system.

7. If a higher priority process arrives while a lower priority process is executing, the current process is preempted immediately.

8. Aging may be applied to promote long-waiting processes to prevent starvation.

9. The scheduling continues until all processes complete execution.

Example:Consider the following processes with Arrival Time (AT) and Burst Time (BT).The system uses **Multilevel Feedback Queue Scheduling (Preemptive)** with the following configuration:

• Queue 1 (Highest Priority) → Round Robin (Time Quantum = 3)
• Queue 2 → Round Robin (Time Quantum = 5)
• Queue 3 (Lowest Priority) → FCFS

Using MLFQ Scheduling, calculate Completion Time (CT),Turnaround Time (TAT),Waiting Time (WT),Average TAT and Average WT.

Process	AT	BT
P1	0	14
P2	2	4
P3	2	9
P4	6	5
P5	15	3

Step 1: Decide Execution Order (Time-wise)

All processes initially enter Queue 1.

Time 0–3
P1 executes in Q1 (remaining 11) → Move to Q2.

Time 2
P2 and P3 arrive (same time) → Insert in Q1 in Process ID order: P2, P3.

Time 3–6
P2 executes (remaining 1) → Move to Q2.

Time 6
P4 arrives → Insert into Q1.

Time 6–9
P3 executes (remaining 6) → Move to Q2.

Time 9–12
P4 executes (remaining 2) → Move to Q2.

Queue 1 becomes empty → Move to Queue 2.

Queue 2 order: P1, P2, P3, P4

Time 12–15
P1 executes in Q2 (3 units used out of 5).

Time 15
P5 arrives → Enters Q1 → Preempts P1 immediately.

Time 15–18
P5 executes in Q1 → Completes.

Back to Q2.

Time 18–20
P1 continues in Q2 (remaining 6 after Q1, used 3 before preemption, so remaining 3 of this quantum, total remaining 8)
After completing full 5 units in Q2, remaining 6 → Move to Q3.

Time 20–21
P2 executes → Completes.

Time 21–26
P3 executes (uses full 5 units, remaining 1) → Move to Q3.

Time 26–28
P4 executes → Completes.

Queue 2 empty → Move to Queue 3.

Queue 3 order: P1, P3

Time 28–34
P1 executes → Completes.

Time 34–35
P3 executes → Completes.

Execution Order:

P1 → P2 → P3 → P4 → P1 → P5 → P1 → P2 → P3 → P4 → P1 → P3

Step 3:Draw Gantt Chart

P1	P2	P3	P4	P1	P5	P1	P2	P3	P4	P1	P3

0 3 6 9 12 15 18 20 21 26 28 34 35

Step 3: Calculate CT, TAT, WT

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	14	34	34	20
P2	2	4	21	19	15
P3	2	9	35	33	24
P4	6	5	28	22	17
P5	15	3	18	3	0

Step 4: Calculate Averages

Average Turnaround Time
= (34 + 19 + 33 + 22 + 3) / 5
= 111 / 5
= 22.2

Average Waiting Time
= (20 + 15 + 24 + 17 + 0) / 5
= 76 / 5
= 15.2

Advantages of Multilevel Feedback Queue Scheduling:Multilevel Feedback Queue Scheduling dynamically adjusts process priorities based on their behavior and it provides good response time for short and interactive processes.

Disadvantages of Multilevel Feedback Queue Scheduling:Multilevel Feedback Queue Scheduling is complex to implement and it may cause high context switching overhead if not properly configured.

LAB EXERCISE

1.**Write a C program to implement Shortest Job First Scheduling.**

a) Non-Preemptive
b) Preemptive (Shortest Remaining Time First)

The program should:

• Accept number of processes n
• Accept Arrival Time (AT) and Burst Time (BT) for each process
• Calculate Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)
• Display the results in tabular format along with average TAT and WT

2.**Write a C program to implement Multilevel Queue Scheduling with the following configuration:**

• Number of Queues = 2
• Queue 1 (Higher Priority) → Priority Scheduling
(Lower number indicates higher priority)
• Queue 2 (Lower Priority) → FCFS Scheduling
• Scheduling between queues is based on Fixed Priority
(Queue 1 has higher priority over Queue 2)
• Processes are permanently assigned to their respective queues.

Implement: a) Non-Preemptive between queues
b) Preemptive between queues

The program should:

• Accept number of processes n
• Accept Arrival Time (AT), Burst Time (BT), Priority (for Queue 1), and Queue Number (1 or 2)
• Calculate CT, TAT, WT
• Display the results in tabular format along with average TAT and WT

3.**Write a C program to implement Multilevel Feedback Queue Scheduling with the following configuration:**

- **Number of Queues = 3**
- **Queue 1 (Highest Priority) → Round Robin (Time Quantum = 2)**
- **Queue 2 → Round Robin (Time Quantum = 4)**
- **Queue 3 (Lowest Priority) → FCFS**

Rules:

- All new processes enter Queue 1
- If a process in Queue 1 uses its full time quantum, it is moved to Queue 2
- If a process in Queue 2 uses its full time quantum, it is moved to Queue 3
- Scheduling between queues is Preemptive
- Higher priority queue can interrupt lower priority queue

The program should:

- Accept number of processes n
- Accept Arrival Time (AT) and Burst Time (BT)
- Calculate CT, TAT, WT
- Display the results in tabular format along with average TAT and WT